Integer ALU Based Computing — ISA Specification v1.0

Instruction Set Architecture for the VFR Integer Processor

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Data Format

1.1 VFR Pair

The atomic data unit. A value and its remainder from division by the batch factor.

[V: i32] [R: i8] → 5 bytes

V is the dividend. R is the remainder. The factor F is declared in the batch header and shared across all values in the batch.

1.2 Batch Header

Every data stream is a sequence of batches. Each batch begins with a 7-byte header:

[F: i16] [count: u32] [depth: u8] → 7 bytes

- **F:** the factor (scale) for this batch, signed, range -32768 to 32767

- **count:** number of values in this batch, 0 = end of stream
- **depth:** number of VFR pairs per value (1 = flat, 2+ = nested polynomial)

Followed by `count $\times$ depth` VFR pairs at uniform stride of `depth $\times$ 5` bytes.

1.3 Stream Format

A program's data is a linear stream of batches:

[F, count, depth] [VR pairs…]

[F, count, depth] [VR pairs…]

[F, count, depth] [VR pairs…]

[0, 0, 0] end of stream (7 zero bytes)

Each batch may have a different factor, count, and depth. Batches are self-describing and independently processable.

1.4 Layout Examples

Single domain, 10 million flat values at scale F=37:

[37, 10_000_000, 1] [v,r] [v,r] [v,r] … ($\times$ 10MM)

[0, 0, 0]

Mixed domains in one stream:

[37, 5_000_000, 1] [v,r] … physics batch

[12, 2_000_000, 1] [v,r] … graphics batch

[8, 500_000, 3] [v,r,v,r,v,r] … audio batch, depth 3

[0, 0, 0]

1.5 Polynomial Nesting

When depth > 1, each value is a sequence of VFR pairs representing base-32 polynomial coefficients:

depth=3: [v0, r0, v1, r1, v2, r2]

reconstructed value = v0 + v1 $\times$ 32 + v2 $\times$ 32^2

with remainders preserved at each level

All values in a batch share the same depth. Stride is fixed within a batch.

---

2. Registers

2.1 General Purpose — Value

V0 - V15 i32 16 value registers

2.2 General Purpose — Remainder

R0 - R15 i8 16 remainder registers

V and R registers are paired by index. V0/R0 through V15/R15 form 16 VFR register pairs.

2.3 Batch Control

BF i16 batch factor (from header)

BC u32 batch count (from header)

BD u8 batch depth (from header)

BI u32 batch index (current position, 0-based)

2.4 Memory

MA u32 memory address register (local only)

MX u32 transfer target address (for DMA)

2.5 Flags

EQ 1-bit set by CMP/CMPR when equal

LT 1-bit set by CMP/CMPR when less than

GT 1-bit set by CMP/CMPR when greater than

OV 1-bit set by RACCUM on remainder overflow

DONE 1-bit set by BNEXT when BI reaches BC

---

3. Instruction Set

35 instructions. All fixed-width. No floating-point. No integer division.

3.1 Arithmetic (5 instructions)

ADD Vd, Va, Vb Vd = Va + Vb

SUB Vd, Va, Vb Vd = Va - Vb

MUL Vd, Va, Vb Vd = Va $\times$ Vb

ADDR Rd, Ra, Rb Rd = Ra + Rb

SUBR Rd, Ra, Rb Rd = Ra - Rb

ADD, SUB, MUL operate on i32 value registers. ADDR, SUBR operate on i8 remainder registers.

3.2 Bitwise (4 instructions)

AND Vd, Va, Vb Vd = Va & Vb

OR Vd, Va, Vb Vd = Va | Vb

XOR Vd, Va, Vb Vd = Va ^ Vb

NOT Vd, Va Vd = ~Va

3.3 Shift (4 instructions)

SHR Vd, Va, imm Vd = Va » imm (logical right shift)

SHL Vd, Va, imm Vd = Va « imm (logical left shift)

SHR5 Vd, Va Vd = Va » 5 (base-32 divide)

SHL5 Vd, Va Vd = Va « 5 (base-32 multiply)

SHR5 and SHL5 are dedicated single-cycle instructions for the base-32 common case.

3.4 VFR Operations (5 instructions)

DECOMP Vd, Rd, Va Vd = Va » 5, Rd = Va & 0x1F

RECOMP Vd, Va, Ra Vd = (Va « 5) | (Ra & 0x1F)

RNORM Rd, Ra Rd = Ra mod 32 (signed, normalized to -16..15)

RACCUM Rd, Ra, Rb Rd = Ra + Rb; set OV flag if |Rd| > 127

SCALE Vd, Va, imm Vd = Va « (imm $\times$ 5)

DECOMP: Single-cycle VFR decomposition. Extracts quotient into Vd and remainder into Rd simultaneously. This is the fundamental VFR operation — one instruction replaces a shift and a mask.

RECOMP: Inverse of DECOMP. Reconstructs a value from quotient and remainder. Single cycle.

RNORM: Normalizes a remainder back into the signed base-32 range after accumulation. Used after a sequence of ADDR/RACCUM operations.

RACCUM: Remainder accumulate with overflow detection. Adds two i8 remainders and sets the OV flag if the result exceeds i8 range. Used in batch reduction operations.

SCALE: Shifts a value by multiple base-32 levels. `SCALE V1, V0, 3` shifts V0 left by 15 bits (3×5). Used when converting between batch factors.

3.5 Compare (2 instructions)

CMP Va, Vb compare i32 values, set EQ/LT/GT flags

CMPR Ra, Rb compare i8 remainders, set EQ/LT/GT flags

Integer comparison. Exact. No epsilon. All lanes evaluate identically.

3.6 Load / Store (6 instructions)

LDV Vd, [MA+offset] load i32 from local memory

LDR Rd, [MA+offset] load i8 from local memory

STV [MA+offset], Va store i32 to local memory

STR [MA+offset], Ra store i8 to local memory

LDVR Vd, Rd, [MA+offset] load VFR pair (5 bytes)

STVR [MA+offset], Va, Ra store VFR pair (5 bytes)

All addresses must fall within the core's local memory range. Access outside this range is a hard fault. There is no exception handler — the core halts.

3.7 Batch Control (3 instructions)

BLOAD [MA] read 7-byte header at MA into BF, BC, BD; reset BI=0

BNEXT BI += 1; set DONE flag if BI == BC

BADDR Vd Vd = MA + 7 + (BI $\times$ BD $\times$ 5)

BLOAD: Initializes a batch. Reads the 7-byte header, populates all batch control registers, and resets the index to zero.

BNEXT: Advances to the next value in the batch. Sets DONE when the batch is exhausted.

BADDR: Computes the memory address of the current value in the batch. The `+ 7` accounts for the header. This is the only address arithmetic instruction needed for batch traversal.

3.8 Transfer / DMA (4 instructions)

TSEND [MA], len, core send len bytes from local MA to target core

TRECV [MA], len receive len bytes into local MA (blocks until complete)

TWAIT block until pending TSEND completes

TDONE signal transfer complete to paired core

TSEND and TRECV are the only mechanism for cross-core data movement. There is no shared memory bus. The transfer is a direct copy from one core's local memory to another's. The sender specifies the destination core ID and a byte length. The receiver must have issued a matching TRECV.

3.9 Control Flow (6 instructions)

JMP addr unconditional jump

JEQ addr jump if EQ flag set

JLT addr jump if LT flag set

JGT addr jump if GT flag set

JDONE addr jump if DONE flag set

HALT stop core execution

No branch prediction. No speculative execution. Jumps are resolved in-order. In batch processing, the typical control flow is a tight loop with JDONE as the only conditional — and it evaluates identically across all values in the batch.

---

4. Instruction Encoding

4.1 Format

All instructions are 32 bits (4 bytes), fixed width, aligned.

[opcode: 6 bits] [operands: 26 bits]

6-bit opcode supports 64 instructions. Current ISA uses 35, leaving 29 slots for future extension.

4.2 Encoding Types

Type A — Three register (arithmetic, bitwise):

[opcode: 6] [Vd: 4] [Va: 4] [Vb: 4] [unused: 14]

Type B — Two register + immediate (shift, scale):

[opcode: 6] [Vd: 4] [Va: 4] [imm: 18]

Type C — VFR pair (DECOMP, RECOMP, LDVR, STVR):

[opcode: 6] [Vd: 4] [Rd: 4] [Va/Ra: 4] [unused: 14]

Type D — Memory (load/store):

[opcode: 6] [reg: 4] [offset: 22]

22-bit offset addresses up to 4MB of local memory per core.

Type E — Transfer (DMA):

[opcode: 6] [core: 10] [len: 16]

10-bit core ID supports up to 1024 cores. 16-bit length supports transfers up to 64KB per instruction.

Type F — Jump:

[opcode: 6] [addr: 26]

26-bit address supports up to 64M instruction addresses (256MB program space).

4.3 Opcode Map

0x00 HALT 0x01 JMP 0x02 JEQ

0x03 JLT 0x04 JGT 0x05 JDONE

0x08 ADD 0x09 SUB 0x0A MUL

0x0B ADDR 0x0C SUBR

0x10 AND 0x11 OR 0x12 XOR

0x13 NOT

0x18 SHR 0x19 SHL 0x1A SHR5

0x1B SHL5

0x20 DECOMP 0x21 RECOMP 0x22 RNORM

0x23 RACCUM 0x24 SCALE

0x28 CMP 0x29 CMPR

0x30 LDV 0x31 LDR 0x32 STV

0x33 STR 0x34 LDVR 0x35 STVR

0x38 BLOAD 0x39 BNEXT 0x3A BADDR

0x3C TSEND 0x3D TRECV 0x3E TWAIT

0x3F TDONE

Opcodes are grouped by function with gaps for future additions within each group.

---

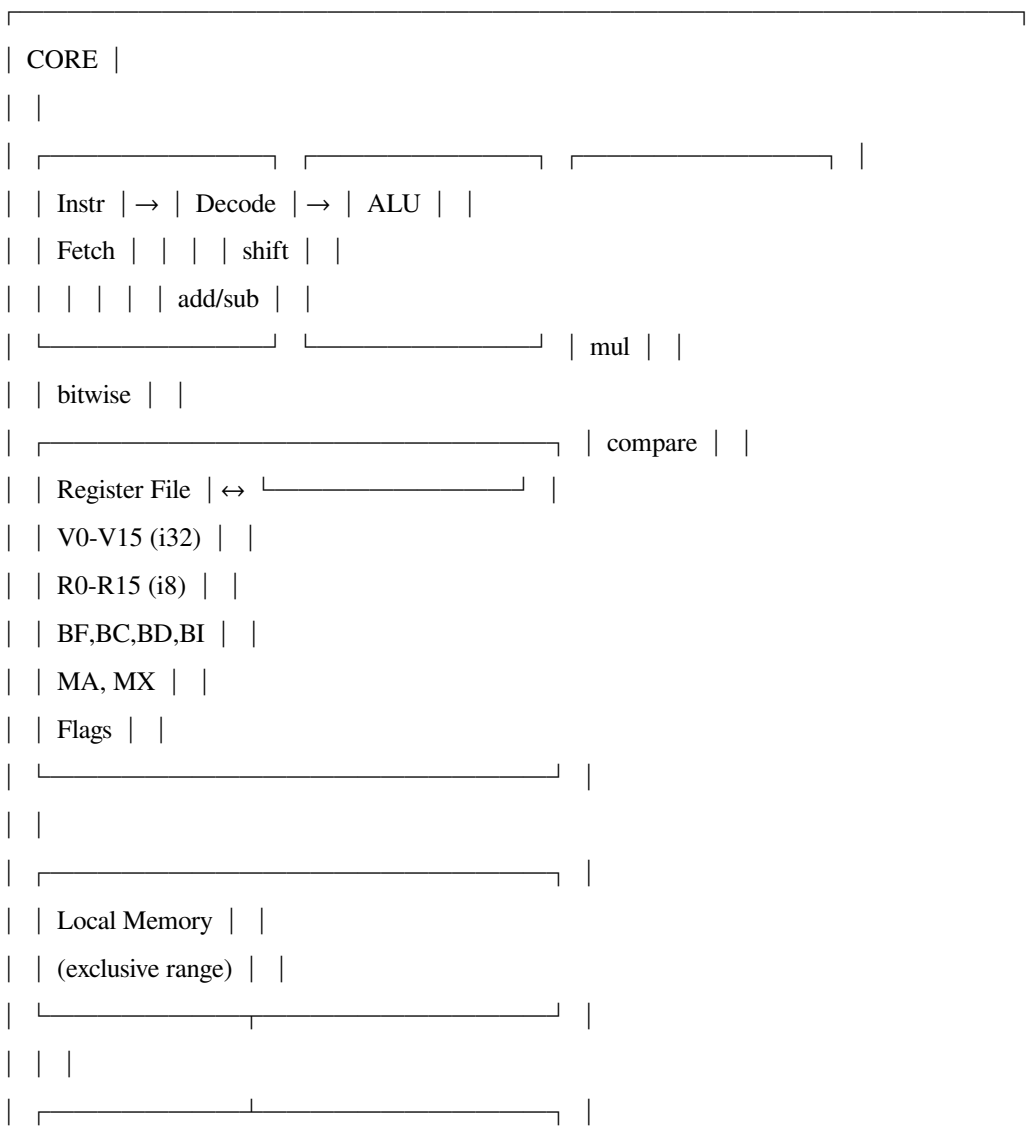
5. Core Architecture

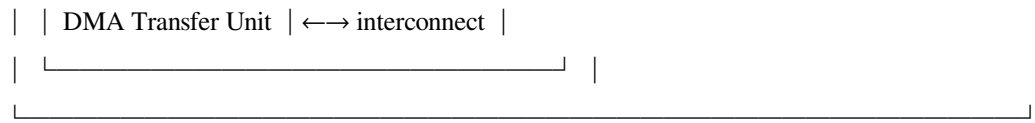
5.1 Pipeline

Fetch → Decode → Execute → Writeback

Four stages. Strictly in-order. No speculation. No branch prediction. No reorder buffer. No register renaming. No out-of-order execution.

5.2 Core Block Diagram





5.3 What Is Not Present

- No floating-point unit
- No floating-point registers
- No IEEE 754 compliance logic
- No integer divider (division is shift)
- No branch predictor
- No speculative execution engine
- No reorder buffer
- No reservation stations
- No register renaming
- No cache coherence controller
- No shared memory interface
- No store buffer snooping
- No memory ordering unit

---

6. Memory Model

6.1 Strict Isolation

Each core is assigned an exclusive, contiguous memory range at boot. A core can only address memory within its own range. There is no global address space visible to cores.

6.2 Local Memory Map

0x000000 - 0x3FFFFF local memory (up to 4MB, configurable)

The 22-bit offset in load/store instructions addresses this full range.

6.3 No Shared State

There is no shared memory, no global bus, no memory-mapped I/O shared between cores. All cross-core communication is explicit DMA transfer.

6.4 Fault Behavior

Any memory access outside the core's assigned range causes an immediate halt. There is no exception handler, no trap, no recovery. The core stops. This is by design — incorrect memory access is a programming error, not a runtime condition to handle.

---

7. Execution Dispatch Model

7.1 Host Controller

A host controller (separate from the processing cores) manages batch dispatch:

1. Read batch header from input stream: [F, count, depth]
2. If count == 0 and F == 0: end of stream, halt
3. Compute partition: per_core = count / num_cores
4. Compute stride: stride = depth $\times$ 5
5. DMA each core's portion to its local memory
6. Signal cores to begin (write header + data to core memory)
7. Cores execute independently
8. Cores signal completion
9. Host collects results via DMA
10. Advance to next batch header

7.2 Core Startup

On signal from host, each core:

1. BLOAD from start of local memory (reads 7-byte header)
2. Enter batch processing loop
3. HALT on completion

7.3 Multi-Batch Pipelining

A core may receive multiple batches in its local memory. After completing one batch, it advances MA past the processed data, executes another BLOAD, and continues. The end-of-stream marker `[0, 0, 0]` (7 zero bytes) terminates processing.

---

8. Typical Programs

8.1 Batch Element-Wise Add

Add corresponding values from two batches. Assumes batch A at offset 0 and batch B at a known offset in local memory.

```
; Load batch A header
BLOAD [0x000000]
; Store batch B base address
; (batch B starts after batch A: 7 + BC × BD × 5)
loop:
JDONE done
; Load value from batch A
BADDR V0
LDVR V1, R1, [V0]
; Load corresponding value from batch B
; (compute B address from BI and known B offset)
LDVR V2, R2, [V0 + batch_b_offset]
; Add values and remainders
ADD V3, V1, V2
ADDR R3, R1, R2
; Store result
STVR [V0], V3, R3
BNEXT
JMP loop
done:
HALT
```

8.2 Scale Transfer Between Batches

Convert values from factor F=12 to factor F=37. The difference is 25 levels = 125 bits of shift. In practice this would be broken into register-width steps.

```
BLOAD [0x000000] ; load source batch (F=12)
loop:
JDONE done
```

BADDR V0

LDVR V1, R1, [V0]

SCALE V1, V1, 25 ; shift left by 25 × 5 = 125 bits

STVR [V0], V1, R1

BNEXT

JMP loop

done:

HALT

8.3 VFR Decomposition Pass

Decompose every value in a batch into quotient and remainder.

BLOAD [0x000000]

loop:

JDONE done

BADDR V0

LDV V1, [V0]

DECOMP V2, R2, V1 ; single cycle: V2 = V1»5, R2 = V1&0x1F

STVR [V0], V2, R2

BNEXT

JMP loop

done:

HALT

---

9. Summary

Property	Specification
Instructions	35
Instruction width	32 bits, fixed
Opcode space	6 bits (64 slots, 29 reserved)
Value registers	16 × i32
Remainder registers	16 × i8
Batch header	7 bytes: i16 factor, u32 count, u8 depth
VFR pair	5 bytes: i32 value, i8 remainder

Property	Specification
Pipeline	4-stage, in-order
Memory per core	Up to 4MB, exclusive
Cross-core data	Explicit DMA only
Max cores addressable	1024 (10-bit core ID)
Division	Not implemented (shift replaces it)
Floating point	Not implemented (VFR replaces it)
Branch prediction	Not implemented (not needed)
Speculation	Not implemented (not needed)
Cache coherence	Not implemented (not needed)

Integer ALU Based Computing — ISA Specification v1.0

Companion document to "Integer ALU Based Computing" architectural paper.

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS MATH-104-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/paper LEX-12-2026